

Andrew Long
Prof Jon McGowan
CSIS-2430-501 - Summer
DATE

Programming Project 2

The problem addressed in this project is how to implement mathematical set operations in software. A set is a collection of distinct elements, and by applying the appropriate logical operations, many common set operations can be performed efficiently by a computer. This is important because modern software frequently compares collections of data to identify shared elements, determine differences, or combine information from multiple sources. In this project, sets are represented using Boolean arrays, allowing logical operators such as AND, OR, and XOR to be used to implement union, intersection, complement, difference, and symmetric difference operations. The project also extends these concepts to multisets, which differ from ordinary sets by allowing the same element to appear more than once. While computers operate using mathematics and logic, many of its users interact with software through natural language. As a result, software must be able to manipulate and understand relations in data beyond simple numerical values and formulas.

For part one of this project, I created an *char* array named *universalSet* with the letters A through L listed alphabetically. I then created two unique subsets named *A and B*, and for each subset created a corresponding Boolean array to represent which elements of the *universalSet* were present in each subset. Using a Boolean array is crucial to this project, as it allows us to easily perform logical operations on each subset to get the results of the needed set operations. By looking at index *i* of each boolean array, and applying the correct logical operator(s) we can iterate through each subset and create a new array that corresponds to the desired set operation. For example, using the logical OR operator allows us to get the Union of two sets. All desired set operations were obtained by using the OR, AND, XOR, and NOT operators.

For part two, rather than a Boolean array I implemented an array of Integers. When dealing with multisets, each element of the array does not need to be unique. For my project I use the same *universalSet*, but created two new unique multisets *A and B* with varied levels of different characters being repeated. With multiples of elements being allowed, moving to an array of Integers allows me to represent how many times each element is present instead of just confirming its existence. The multiset operations we conducted required a much simpler strategy to approach, as I just iterated through each multiset and applied the max and min methods from Java's Math library as well as basic addition and subtraction.

To verify the results of my program, I created some junit test cases. For the functions in part 1 relating to subsets, each of the expected results was manually fact checked on paper by myself using Venn Diagrams to assist in my checking. The multiset operations I used a simple excel sheet to ensure the math was correct. For some edge cases I created a situation where

the multiset difference would normally result in a negative value to ensure we did not drop below zero. Another edge case I tested was the union operator against a completely empty set, which should provide the original set unchanged.

Here are the results of my program:

PART 1		
Operation	Bit String	Elements
Set A	110100010111	{A, B, D, H, J, K, L}
Set B	110000110101	{A, B, G, H, J, L}
NOT(A) – Complement of A	1011101000	{C, E, F, G, I}
$A \cup B$ – Union	110100110111	{A, B, D, G, H, J, K, L}
$A \cap B$ – Intersection	110000010101	{A, B, H, J, L}
$A - B$ – Difference	100000010	{D, K}
$A \oplus B$ – Symmetric Difference	100100010	{D, G, K}

PART 2												
Operation	A	B	C	D	E	F	G	H	I	J	K	L
Multiset A	8	1	0	4	0	2	1	2	3	8	2	9
Multiset B	6	0	2	1	1	1	4	2	3	6	4	5
Union ($A \cup B$)	8	1	2	4	1	2	4	2	3	8	4	9
Intersection ($A \cap B$)	6	0	0	1	0	1	1	2	3	6	2	5
Difference ($A - B$)	2	1	0	3	0	1	0	0	0	2	0	4
Sum ($A + B$)	14	1	2	5	1	3	5	4	6	14	6	14

Reviewing the project I created, and the proposal of using an overloaded method, I would need to retweak my program quite a bit to ensure the results are accurate. The coding structure I used for Boolean arrays in the sets, and the Integer arrays for the multiset, these methods use completely different rules and logic to create their results. So as my code stands now, I would not be able to accommodate different data types as each of those data types has separate rules that need to be implemented. As for the ability to determine what type of set to use based on a user's natural-language query, that could prove difficult. While someone experienced with sets may be able to word a query in a way to include words that help in suggesting the correct type, words such as unique, or frequency; the average user would most likely not include those in their request. The data types I designated for this project were Boolean and Integer. If we were to try and move the information from the Boolean array into the Integer array, we would not lose the data on the array itself. That would not work the other way

around, as a Boolean array would not be able to properly account for the multiples of the set and data would be lost.

Taking a step back, reviewing my project, and reviewing the paragraph above I can examine how one might attempt to code an interface that would allow users to type in natural-language queries. The system itself would first need to be able to detect whether the user intends to work with a set or a multiset. Because the conversion of datatypes does not work in both directions, ensuring the system would create the correct datatype before any set operations would be vital. To address these challenges, the system could use overloaded methods as a common data structure, the method would need to be able to implement the correct logic and rules on the input depending on what array is detected. The implication and challenge of a system like this is that our natural language can often be highly ambiguous while the structure and rules one would need to implement would require explicit wording and definitions. Without a trained user, or a highly advanced logic system to better assist in anticipating the users request, developing said system would prove very difficult.

This project was an interesting one for me, as it was enjoyable to take concepts I previously learned in lower level math classes and first programming courses and mix them with the idea of data structures. Using simple logical operators to implement set operations showed useful fundamentals in programming and mathematics and still can do a lot of work in progressing with higher level concepts. Seeing the usefulness of translating a set into a Boolean String and then using binary math was another fun part of this project that brought a lot of concepts that have been built upon over my time pursuing my degree in computer science. Beyond the work of the project, the prompts that helped guide this report poised some interesting comparisons on how our natural language can differ so widely from the precise language and logic that our programs need in order to function correctly.